

20. Valid Parentheses

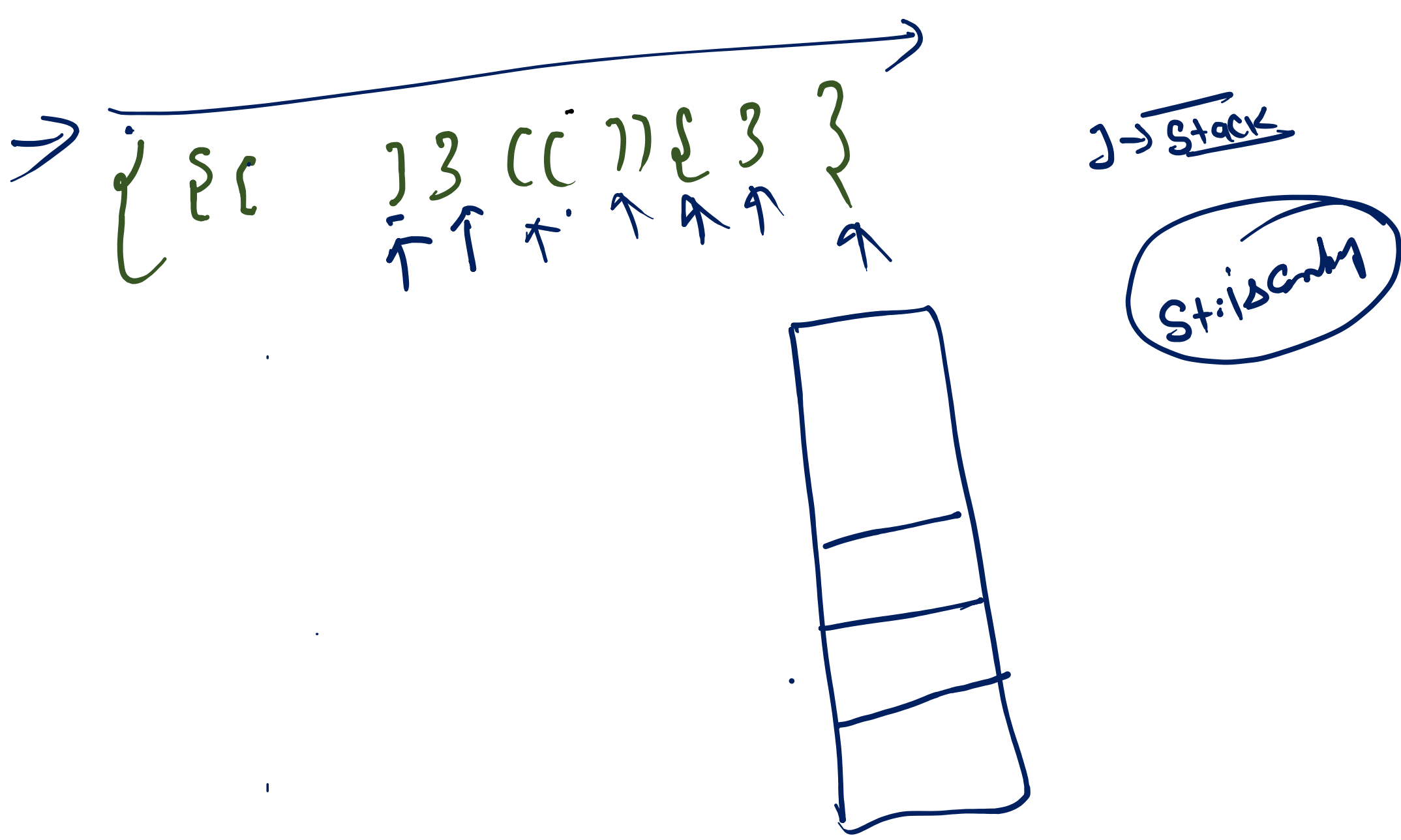
Solved

Easy Topics Companies Hint

Given a string `s` containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

- 1. Open brackets must be closed by the same type of brackets.
- 2. Open brackets must be closed in the correct order.
- 3. Every close bracket has a corresponding open bracket of the same type.



```
public static boolean isValid(String s) {
    Stack<Character> st = new Stack<>();
    for (int i = 0; i < s.length(); i++) {
        char ch = s.charAt(i);
        if (ch == '(' || ch == '{' || ch == '[') {
            st.push(ch);
        } else {
            if (st.isEmpty()) {
                return false;
            } else if (ch == ']' && st.peek() != '[') {
                return false;
            } else if (ch == '}' && st.peek() != '{') {
                return false;
            } else if (ch == ')' && st.peek() != '(') {
                return false;
            }
            st.pop();
        }
    }
    return st.isEmpty();
}
```

You are given a 0-indexed string `pattern` of length `n` consisting of the characters `'I'` meaning increasing and `'D'` meaning decreasing.

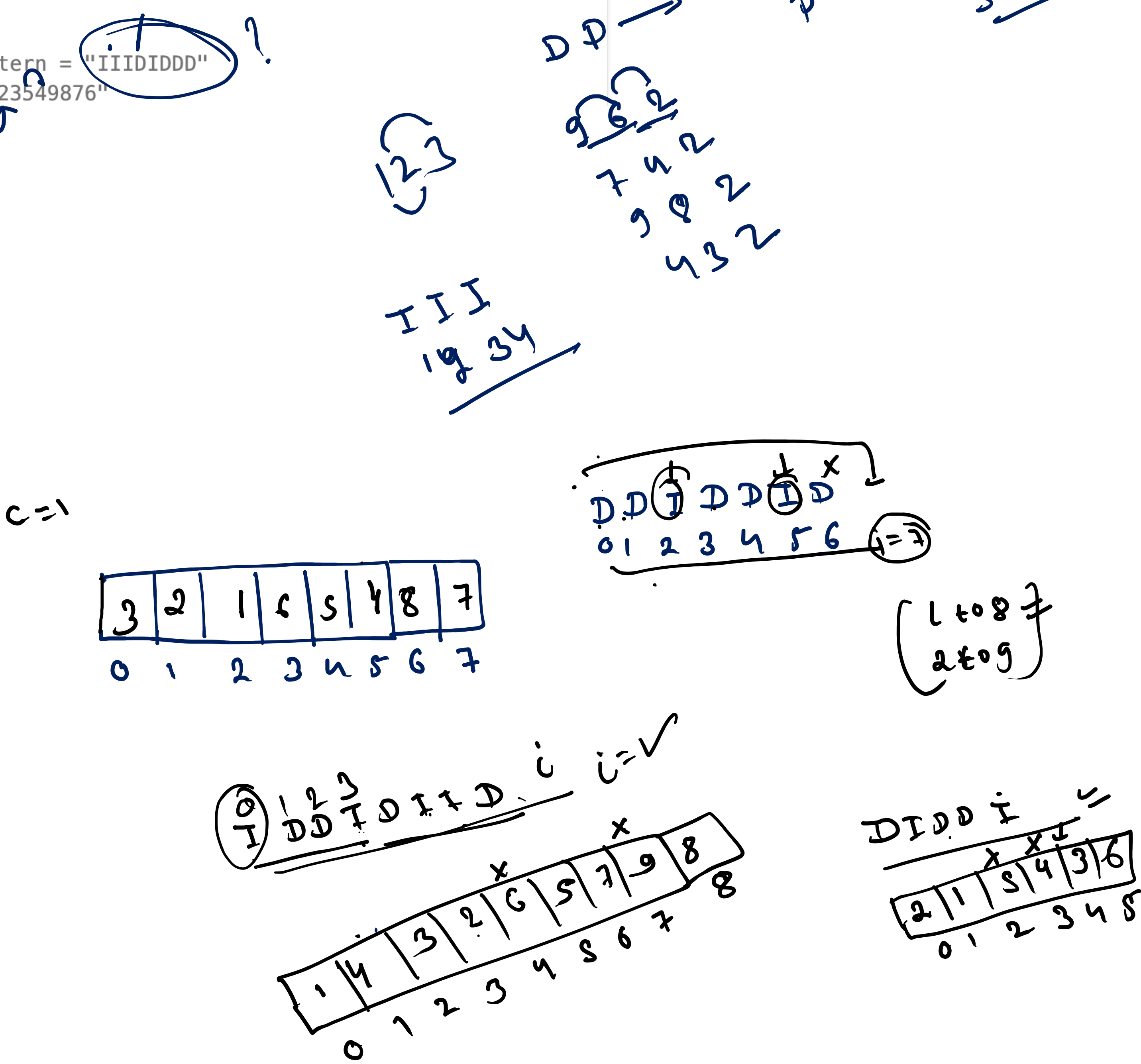
A 0-indexed string `num` of length `n + 1` is created using the following conditions:

- `num` consists of the digits `'1'` to `'9'`, where each digit is used at most once.
- If `pattern[i] == 'I'`, then `num[i] < num[i + 1]`.
- If `pattern[i] == 'D'`, then `num[i] > num[i + 1]`.

Return the lexicographically smallest possible string `num` that meets the conditions.

Example 1:

Input: `pattern = "IIDIDDDD"`
Output: `"123549876"`



```
public static String smallestNumber(String pattern) {
    int[] ans = new int[pattern.length() + 1];
    int c = 1;
    for (int i = 0; i <= pattern.length(); i++) {
        if (i == pattern.length() || pattern.charAt(i) == 'I') {
            ans[i] = c++;
        } else {
            S.push(c);
        }
    }
    while (S.isEmpty() == false) {
        ans[i] = S.pop();
        i++;
    }
}
```

